

Logistic Regression

Aim

To implement a Logistic Regression model in Python to classify data points and evaluate its performance using a classification report.

Procedure

1. Import the required libraries such as pandas, numpy, and sklearn.
2. Load the dataset into the program and examine it to understand the features and target variable.
3. Select the relevant independent variable(s) and the dependent variable for classification.
4. Initialize a loop to iterate through multiple random states to check for model performance variation.
5. In each iteration, split the dataset into training and testing sets using the current random state.
6. Create a Logistic Regression model using sklearn.
7. Train the model using the training dataset.
8. Evaluate the model's accuracy on both the training and testing datasets using the score() method.
9. Compare the training and testing scores for each random state and identify the one where the test score is higher than the training score.
10. Display the best random state and its corresponding accuracy scores to confirm the model's generalization capability.

```
import pandas as pd
import numpy as np
```

```
df=pd.read_csv('SNA.csv')
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased	
0	15624510	Male	19	19000	0	
1	15810944	Male	35	20000	0	
2	15668575	Female	26	43000	0	
3	15603246	Female	27	57000	0	
4	15804002	Male	19	76000	0	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.isnull().sum()
```

	0
User ID	0
Gender	0
Age	0
EstimatedSalary	0
Purchased	0

```
dtype: int64
```

```
features=df.iloc[:,[2,3]]
label=df.iloc[:,4]
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
for i in range(1,401):
    x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=i)
    model=LogisticRegression()
    model.fit(x_train,y_train)
    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)
    if test_score>train_score:
        print("Test {} Train{} Random State {}".format(test_score,train_score,i))
```

```

Test 0.875 Train0.846875 Random State 305
Test 0.8625 Train0.834375 Random State 305
Test 0.9125 Train0.8375 Random State 306
Test 0.875 Train0.846875 Random State 308
Test 0.9 Train0.84375 Random State 311
Test 0.8625 Train0.834375 Random State 313
Test 0.9125 Train0.834375 Random State 314
Test 0.875 Train0.8375 Random State 315
Test 0.9 Train0.846875 Random State 317
Test 0.9125 Train0.821875 Random State 319
Test 0.8625 Train0.85 Random State 321
Test 0.9125 Train0.828125 Random State 322
Test 0.85 Train0.846875 Random State 328
Test 0.85 Train0.8375 Random State 332
Test 0.8875 Train0.853125 Random State 336
Test 0.85 Train0.8375 Random State 337
Test 0.875 Train0.840625 Random State 343
Test 0.8625 Train0.84375 Random State 346
Test 0.8875 Train0.83125 Random State 351
Test 0.8625 Train0.85 Random State 352
Test 0.95 Train0.81875 Random State 354
Test 0.8625 Train0.85 Random State 356
Test 0.9125 Train0.840625 Random State 357
Test 0.8625 Train0.8375 Random State 358
Test 0.85 Train0.840625 Random State 362
Test 0.9 Train0.84375 Random State 363
Test 0.8625 Train0.853125 Random State 364
Test 0.9375 Train0.821875 Random State 366
Test 0.9125 Train0.840625 Random State 369
Test 0.8625 Train0.853125 Random State 371
Test 0.925 Train0.834375 Random State 376
Test 0.9125 Train0.828125 Random State 377
Test 0.8875 Train0.85 Random State 378
Test 0.8875 Train0.85 Random State 379
Test 0.8625 Train0.840625 Random State 382
Test 0.8625 Train0.859375 Random State 386
Test 0.85 Train0.8375 Random State 387
Test 0.875 Train0.828125 Random State 388
Test 0.85 Train0.84375 Random State 394
Test 0.8625 Train0.8375 Random State 395
Test 0.9 Train0.84375 Random State 397
Test 0.8625 Train0.84375 Random State 400

```

```

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=376)
model=LogisticRegression()
model.fit(x_train,y_train)

```

▼ LogisticRegression ⓘ ?

LogisticRegression()

```

print(model.score(x_train,y_train))
print(model.score(x_test,y_test))

```

```

0.834375
0.925

```

Start coding or [generate](#) with AI.

```

from sklearn.metrics import classification_report
print(classification_report(label,model.predict(features)))

```

	precision	recall	f1-score	support
0	0.86	0.91	0.89	257
1	0.83	0.74	0.78	143
accuracy			0.85	400
macro avg	0.85	0.83	0.84	400
weighted avg	0.85	0.85	0.85	400

Result:

Hence, a Python program to implement Logistic Regression has been executed successfully, and the random state with a higher test score than the training score has been identified, ensuring better model generalization.